Deep Learning

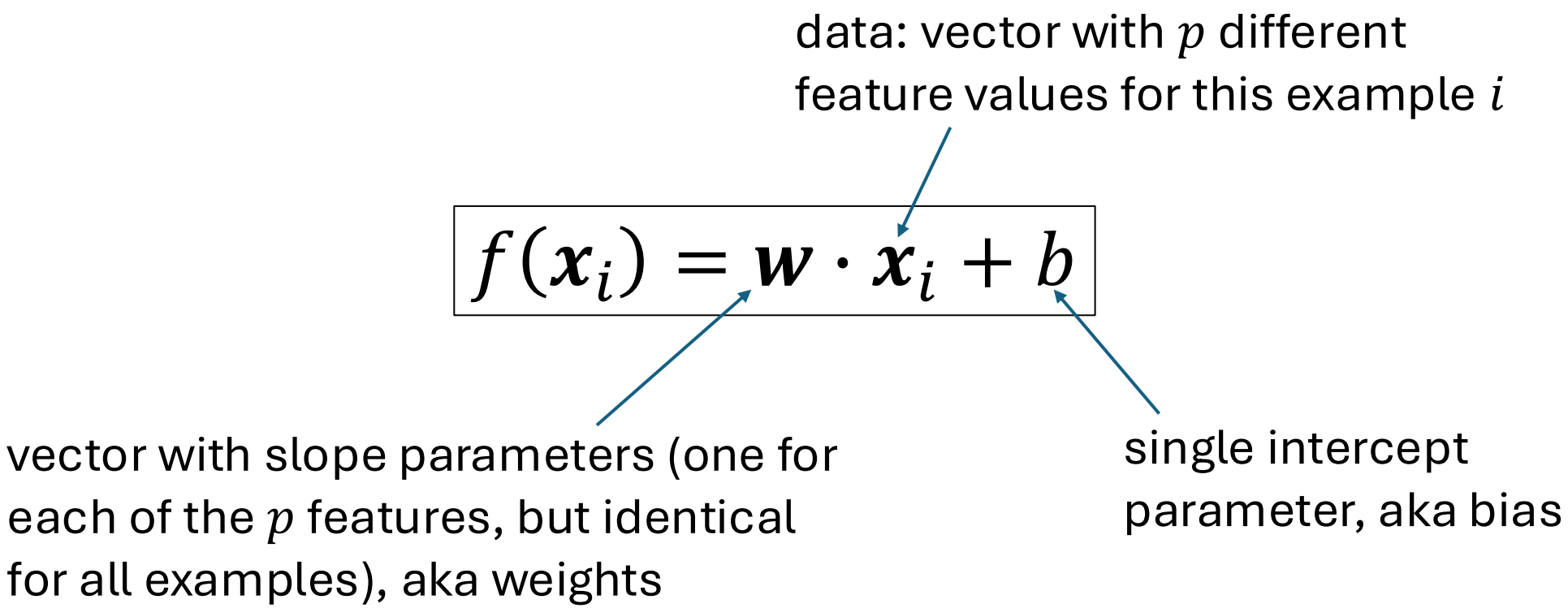
Fundamentals of Artificial Intelligence

General Recipe of Statistical Learning

ML algorithm by combining:

- **model** (e.g., linear function & Gaussian distribution)
- **objective function** (e.g., squared residuals)
- **optimization algorithm** (e.g., gradient descent)
- **regularization** (e.g., L2, dropout)

Linear Model



Loss Function

loss function L: expressing deviation between prediction $\hat{y}$ and target y

$$L(y_i, \hat{y}_i) = L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{w})\big)$$

with $\boldsymbol{w}$ corresponding to parameters of model $f(\boldsymbol{x}_i)$

(for the sake of clarity, include bias b in $\boldsymbol{w}$)

prime example: squared residuals for regression problems

$$L(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

Cost Function

averaging losses over empirical training data set:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} L\big(y_i, f(\boldsymbol{x}_i; \boldsymbol{w})\big)$$

cost function to be minimized according to model parameters $\boldsymbol{w}$

→ objective function

Cost Minimization

minimize training costs $\mathcal{L}(\boldsymbol{w})$ according to model parameters $\boldsymbol{w}$:

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = 0$$

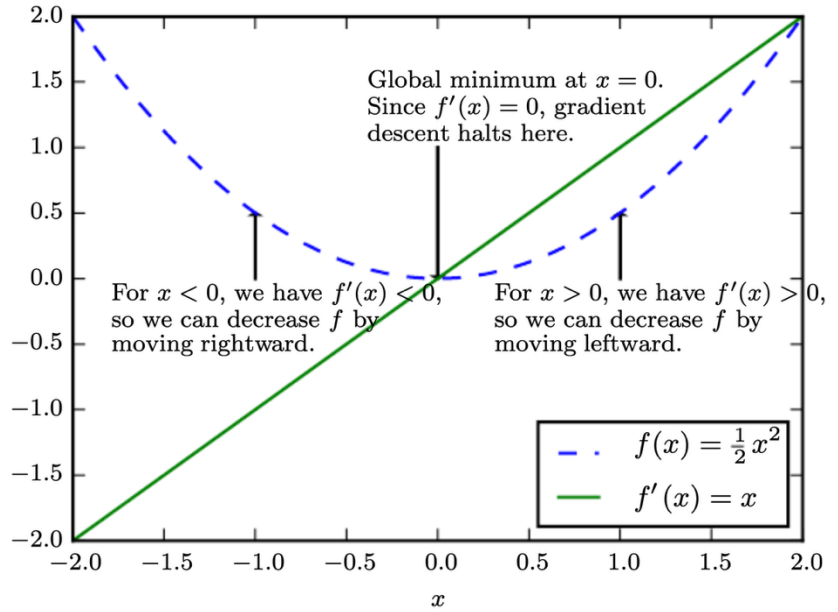
for mean squared error (aka least squares method):

$$\nabla_{\boldsymbol{w}} \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i; \boldsymbol{w}) \right)^2 = 0$$

Gradient Descent

typically no closed-form solution to minimization of cost function: $\nabla_{\boldsymbol{w}}\mathcal{L} = 0$

→ need for numerical methods



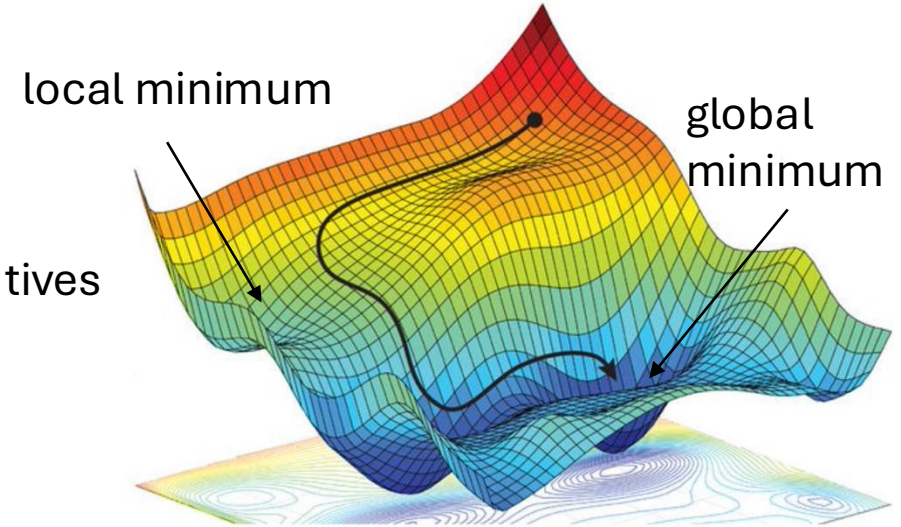
source

popular choice: gradient descent

iteratively moving in direction of steepest descent (negative gradient): $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\nabla_{w}\mathcal{L}$

step size (learning rate)

vector containing all partial derivatives



L^2 Regularization (Ridge Regression)

add **penalty term** on parameter L^2 norm to cost function

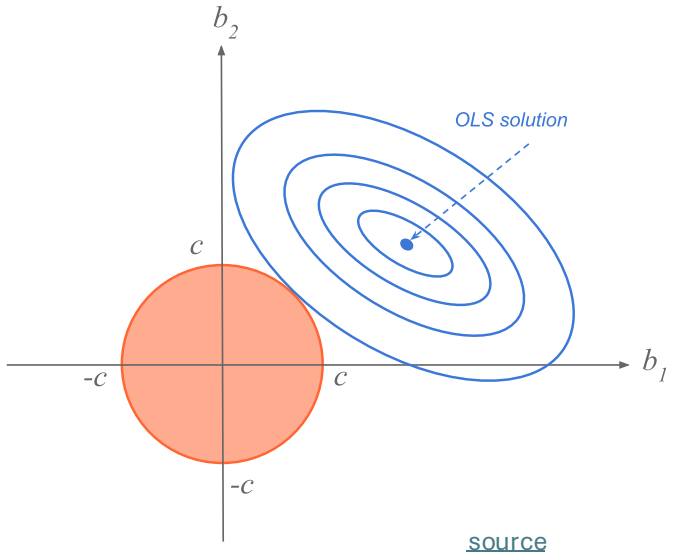
$$\tilde{\mathcal{L}}(\boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \lambda \cdot \boldsymbol{w}^{\mathrm{T}}\boldsymbol{w}$$

hyperparameter controlling strength of regularization

aka **weight decay** in neural networks

corresponds to imposing constraint on parameters to lie in L^2 region (size controlled by λ)

goal: reduce overfitting



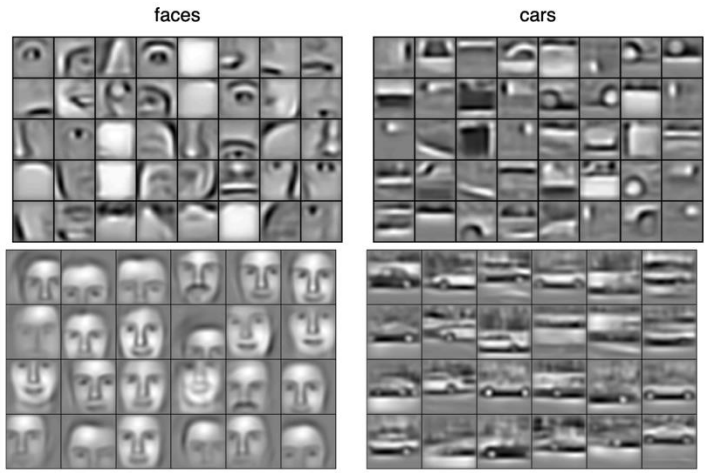
source

Ladder of Generalization

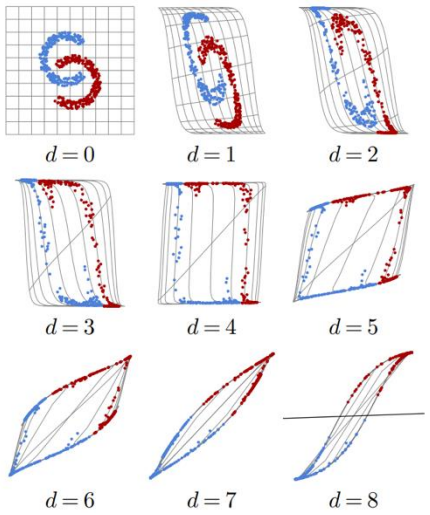
classic ML: feature engineering

deep learning: feature learning

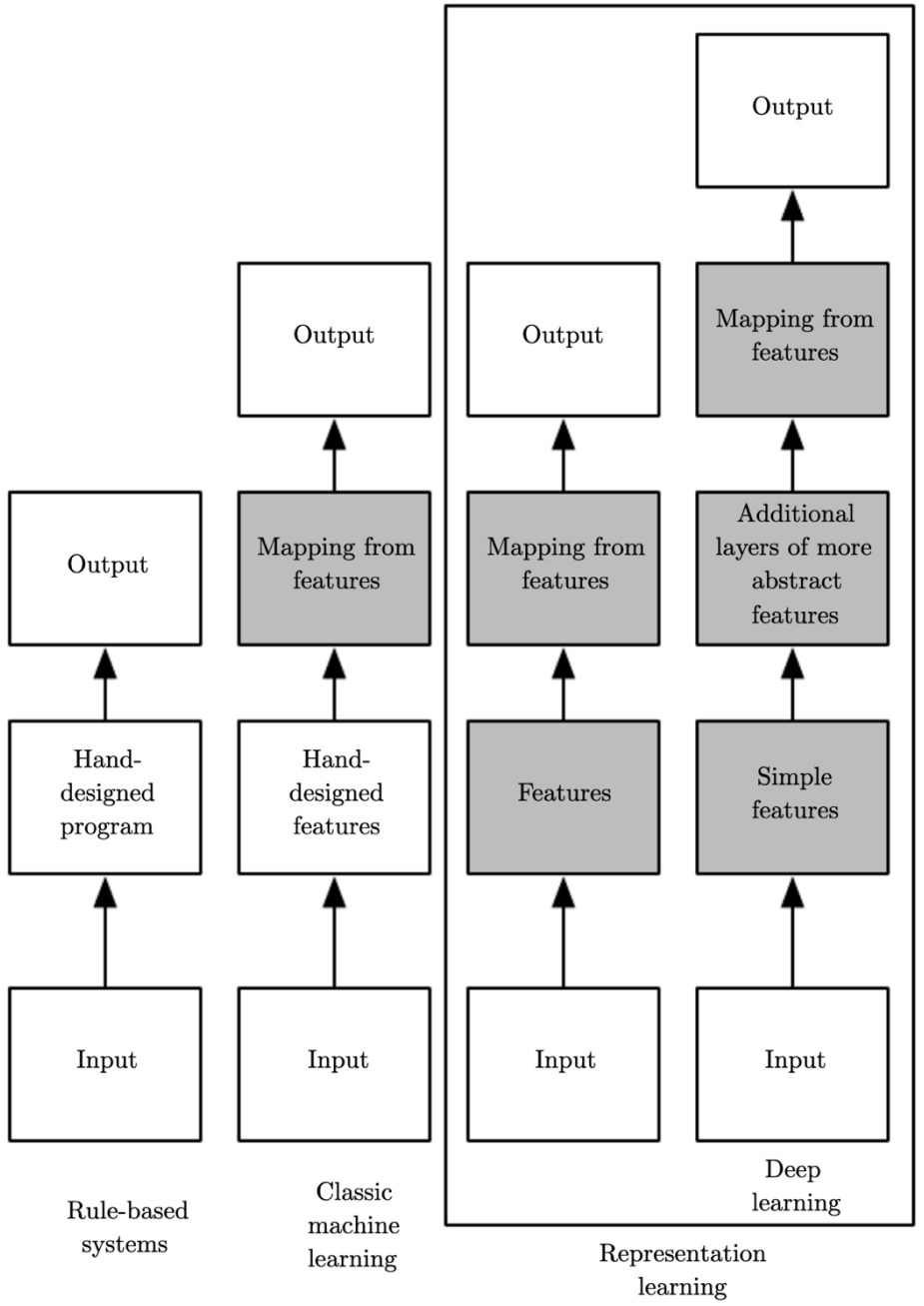
(hierarchy of concepts learned from raw data in deep graph with many layers)



source



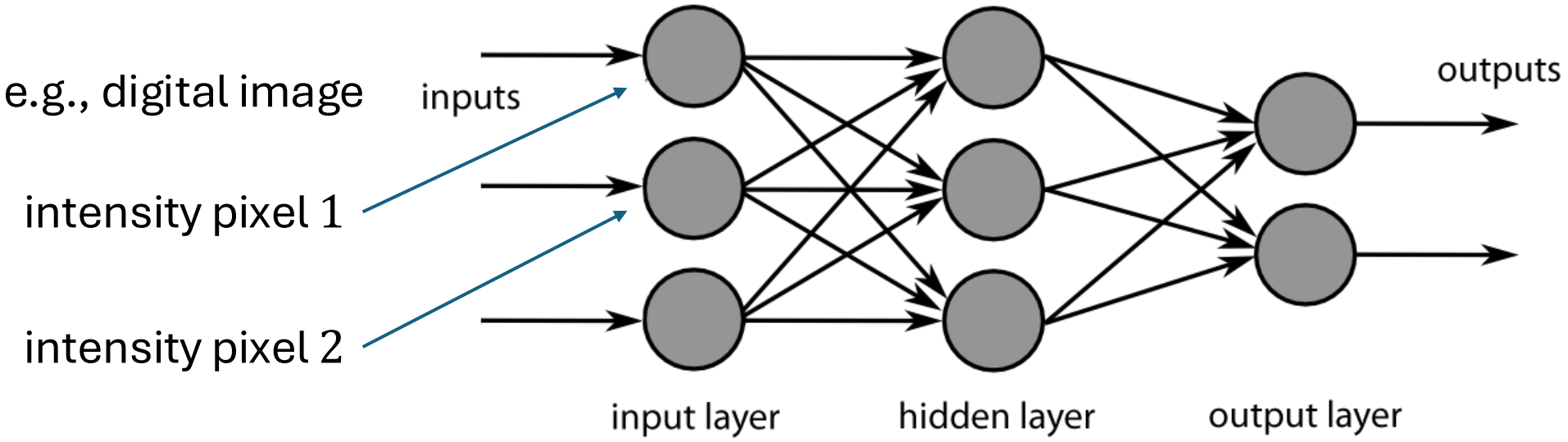
source



source

Neural Networks

idea: powerful algorithm by combining many linear building blocks



each node exchanges information only with directly connected nodes
→ parallel computation

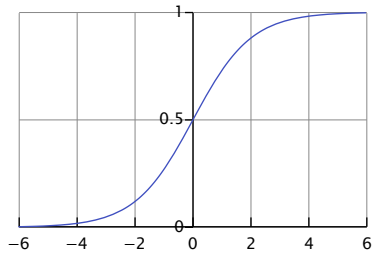
Artificial Neuron

linear model with parameters called weights $\boldsymbol{w}$ (including bias, not shown for simplicity)

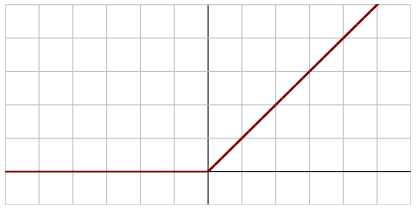
non-linear via (differentiable) activation function on top of linear model

examples:

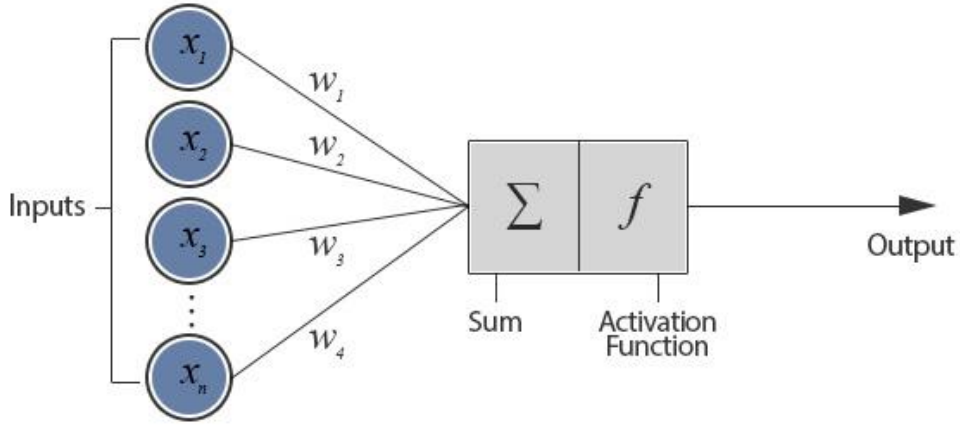
- sigmoid



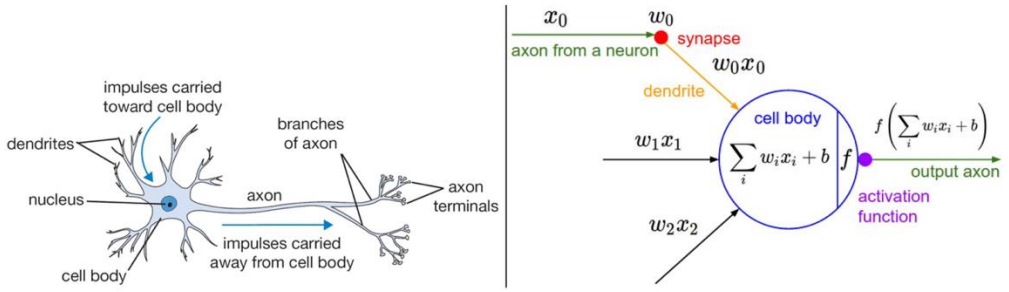
- ReLU (Rectified Linear Unit)



artificial neuron (node in neural network):

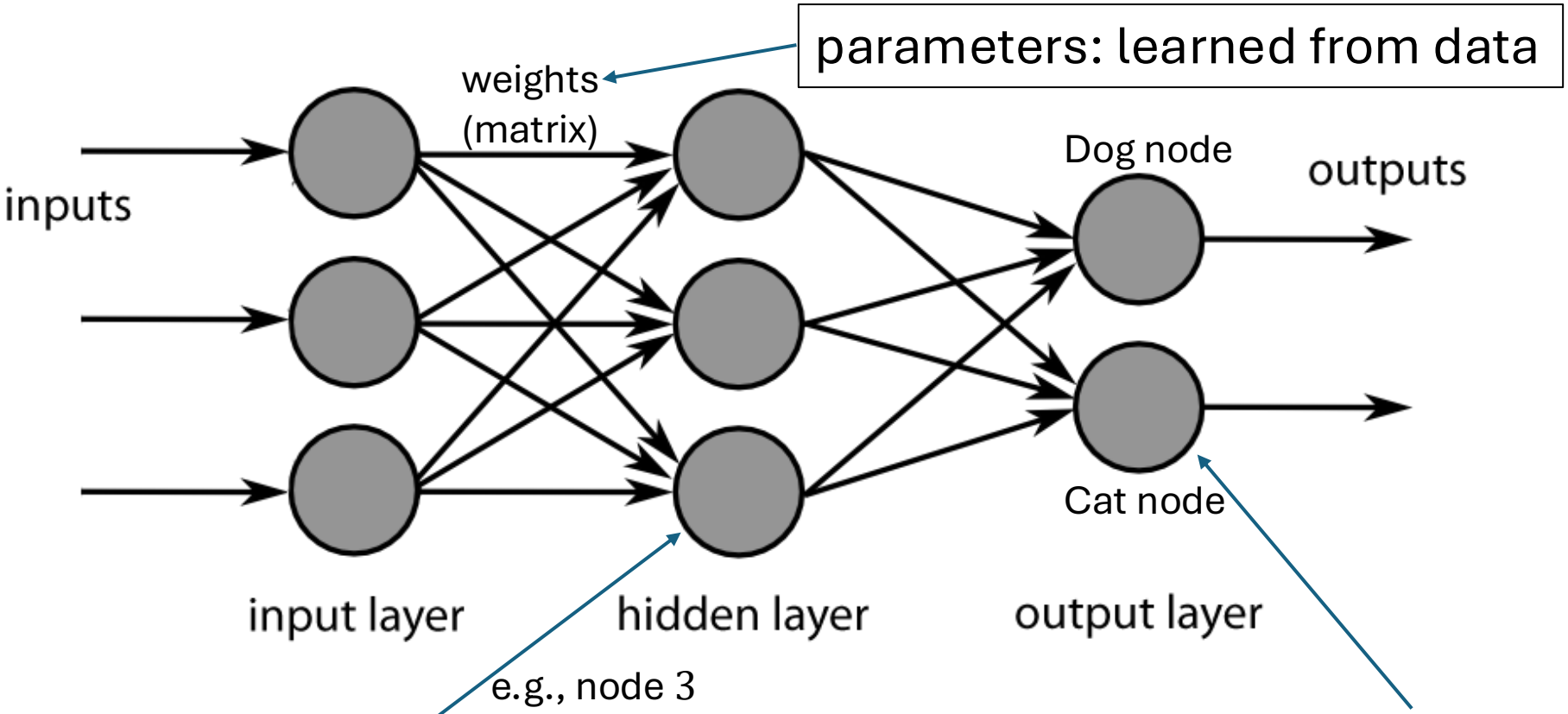


inspired from biological neurons:

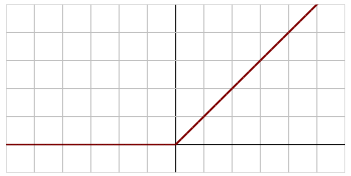


source

Neural Network as Nonlinear Function



linear transformation with nonlinear activation functions



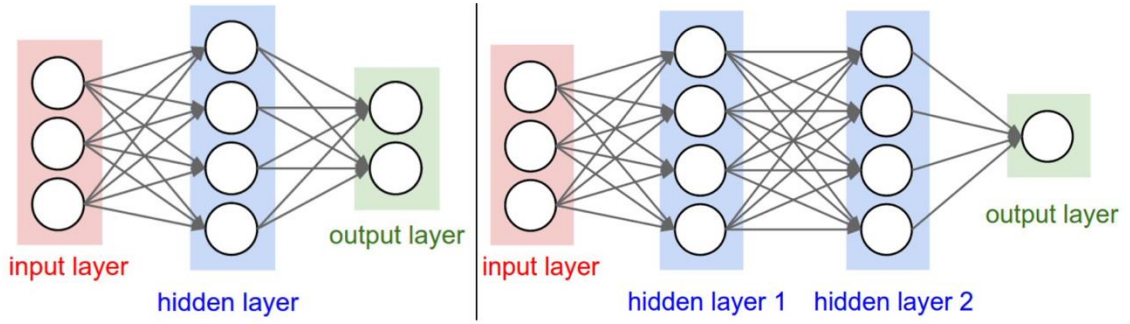
$$\left(\overrightarrow{w_3} \cdot \vec{x}\right)$$

probability for „cat“
(softmax instead of activation function)

Multi-Layer Perceptron (MLP)

fully-connected feed-forward network with at least one hidden layer (universal function approximator)

binary classification



regression

toward deep learning: add hidden layers

more layers (depth) more efficient than just more nodes (width): less parameters needed for same function complexity

Classification Networks

cross-entropy loss (one output node for each class k):

$$L(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

using softmax on output nodes:

$$\hat{y}_k = \frac{e^{o_k}}{\sum_{l=1}^{K} e^{o_l}}$$

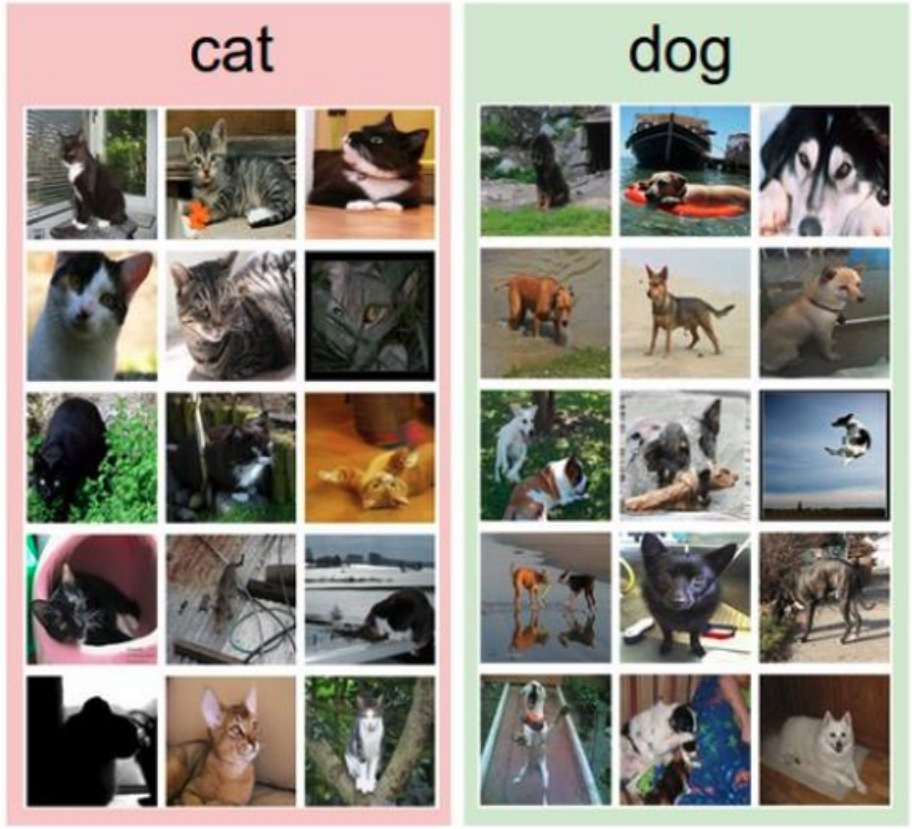
→ prediction of probabilities

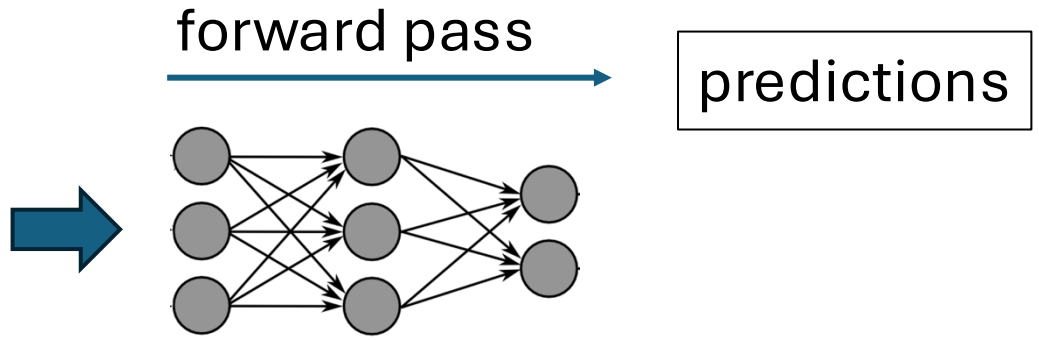
regression networks:

- squared error loss: $L(y, \hat{y}) = (y - \hat{y})^2$
- just one output node
- no function on output → prediction of real numbers

Training with Labeled Images

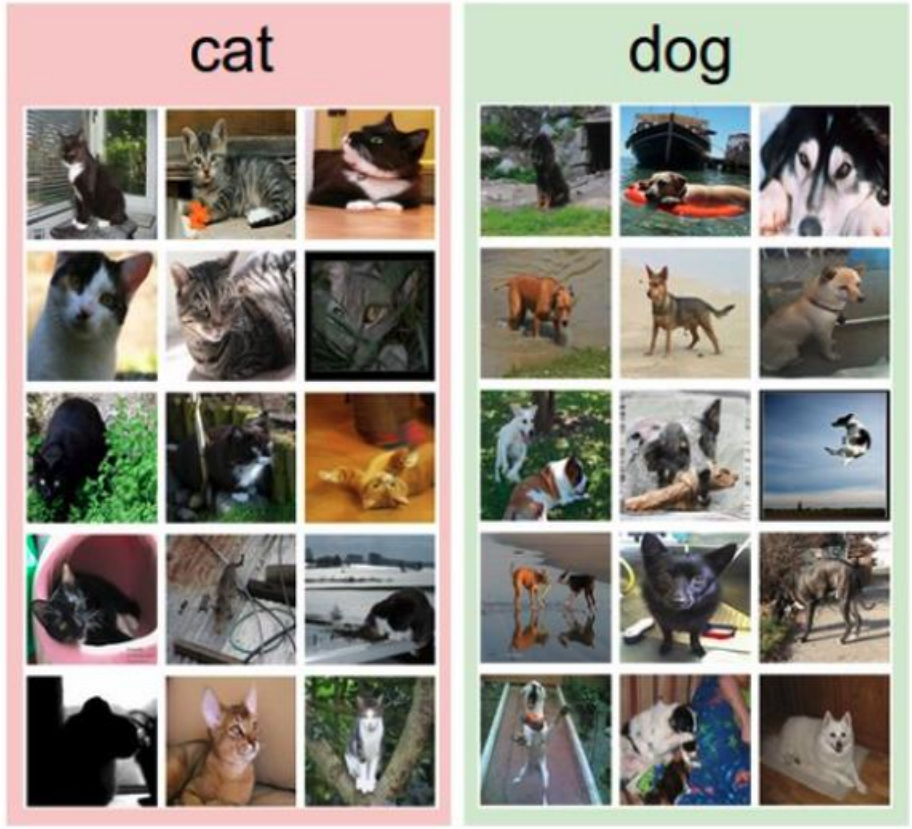
label:

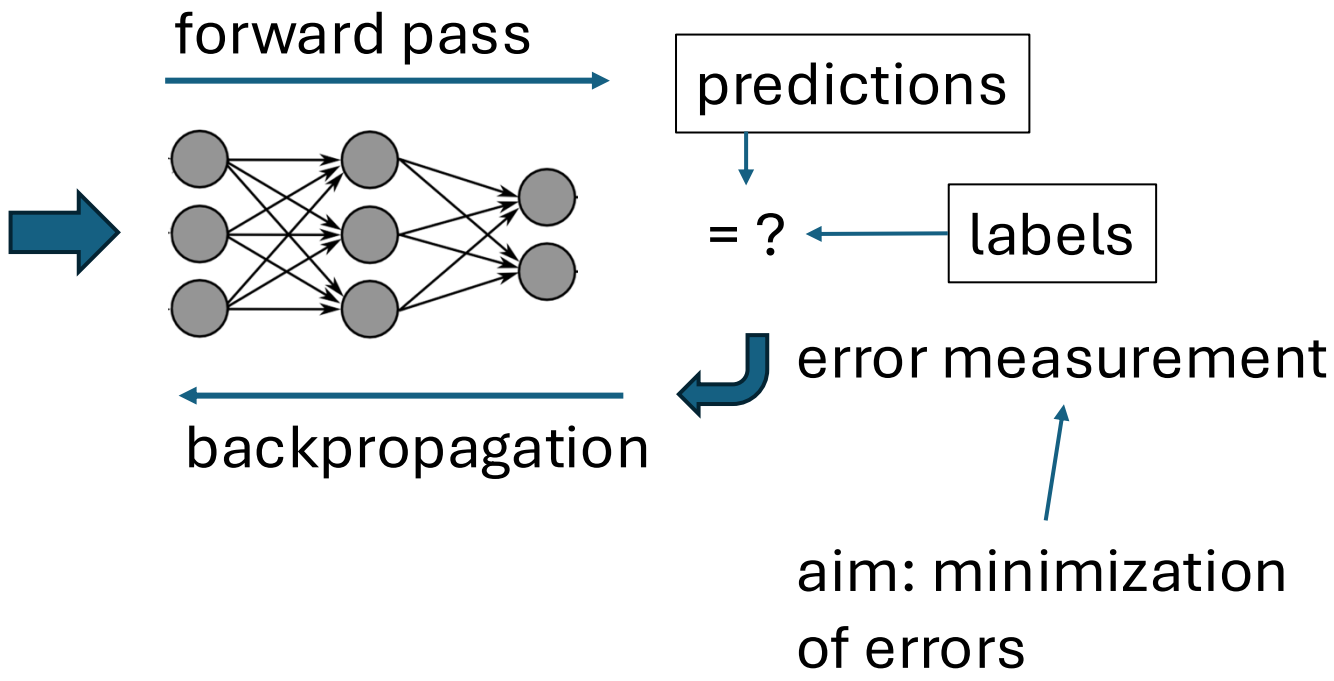




Training with Labeled Images

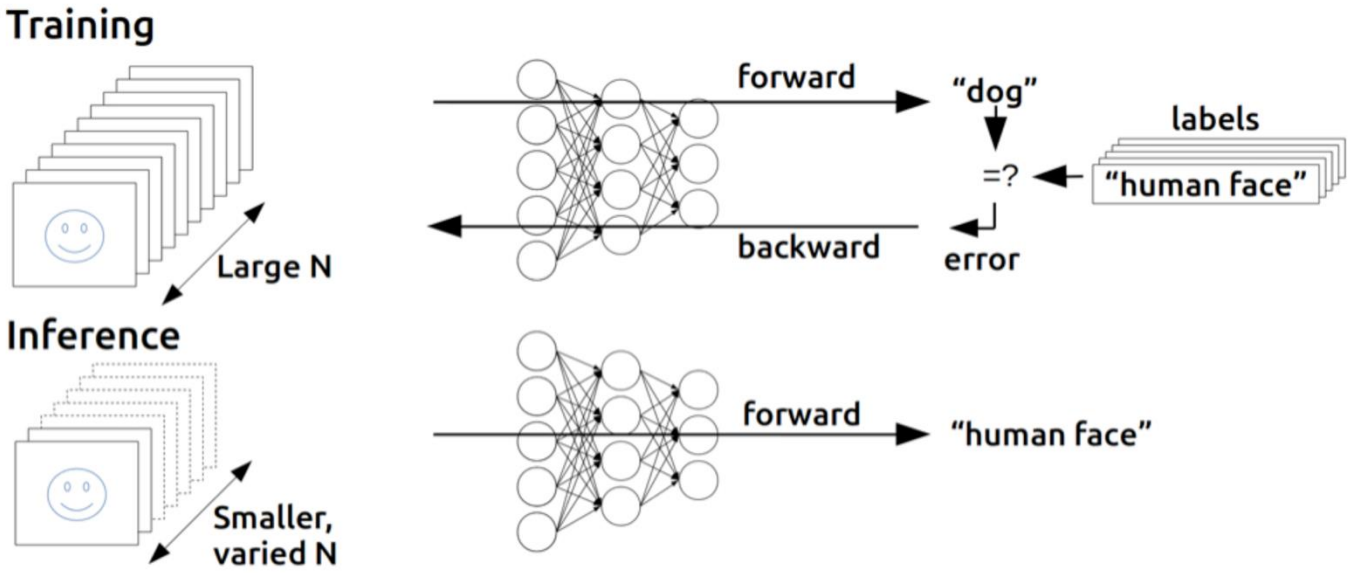
label:





Find Gradients for (Deep) Neural Networks

backpropagation of errors through network layers via chain rule of calculus (enables learning of deep neural networks)



source

forward pass:

- fix current weights
- compute predictions

backward pass:

- compute errors
- calculate gradients (backpropagation of errors)
- update weights accordingly (gradient descent)

Training: Iterative Adjustment of Weights

```
for epoch in range(epochs):
    for X, y in mini_batches:
        pred = model(X)
        cost = loss(pred, y)
        cost.backward()
        optimizer.step()
```

quantification of error (here: cross entropy)

chain rule to calculate gradient (direction) of cost function with respect to weights

one step in direction of steepest descent: (stochastic) gradient descent

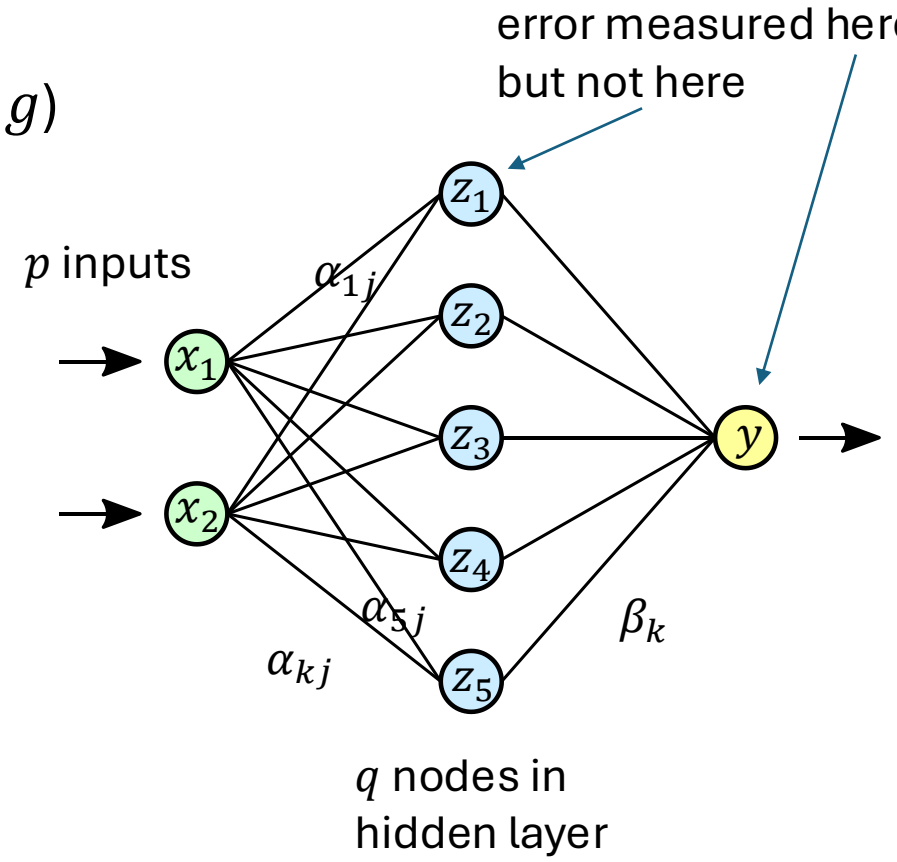
Example WOLOG

- regression (squared error loss, identity output function g)
- with one hidden layer ($\boldsymbol{w}$: $\boldsymbol{\alpha}, \boldsymbol{\beta}$)

$$\hat{y}_i = f(\boldsymbol{x}_i; \boldsymbol{w}) = g(\boldsymbol{z}_i; \boldsymbol{\beta}) = \sum_{k=0}^{q} \beta_k z_{ik}$$

$$z_{ik} = h(\boldsymbol{x}_i; \boldsymbol{\alpha}_k) = h\left(\sum_{j=0}^{p} \alpha_{kj} x_{ij}\right)$$

activation function



cost function:

$$\mathcal{L}(\boldsymbol{w}) = \sum_{i=1}^{n} L_i(y_i, \hat{y}_i; \boldsymbol{w}) = \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i;\ \boldsymbol{\alpha}, \boldsymbol{\beta})\right)^2$$

Example WOLOG

gradients:

$$\frac{\partial L_i}{\partial \beta_k} = -2\,(y_i - \hat{y}_i) z_{ik} = \delta_i z_{ik}$$

$$\frac{\partial L_i}{\partial \alpha_{kj}} = -2\,(y_i - \hat{y}_i)\beta_k h_k' \left(\sum_{j=0}^{p} \alpha_{kj} x_{ij} \right) x_{ij} = s_{ik} x_{ij}$$

backpropagation equations: $s_{ik} = h_k' \left(\sum_{j=0}^{p} \alpha_{kj} x_{ij} \right) \beta_k \delta_i$

use errors of later layers to calculate errors of earlier ones (avoiding redundant calculations of intermediate terms)

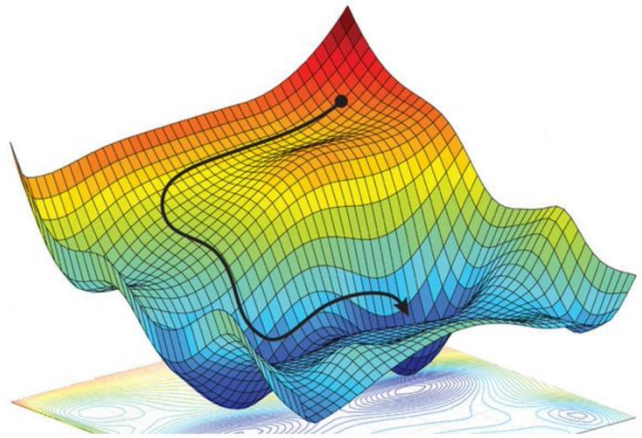
Using Gradients for Iterative Learning

use gradients found via backpropagation to iteratively update weights (gradient descent):

$$\beta_k^{(r+1)} = \beta_k^{(r)} - \eta_r \sum_{i=1}^{n} \frac{\partial L_i}{\partial \beta_k^{(r)}} \qquad \alpha_{kj}^{(r+1)} = \alpha_{kj}^{(r)} - \eta_r \sum_{i=1}^{n} \frac{\partial L_i}{\partial \alpha_{kj}^{(r)}}$$

- adaptive learning rate η_r: learning rate often adjusted per iteration
- weight initialization: choose small random weights as starting values to break symmetry (typically using some heuristics)

Stochastic Gradient Descent (SGD)



step size

cost function

weight updates: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} \mathcal{L}$

all training examples

options:

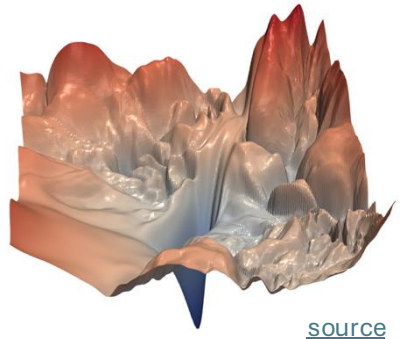
- $\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$ batch gradient descent (whole training dataset)
- $\mathcal{L} = L_i$ stochastic gradient descent (single example)
- $\mathcal{L} = \frac{1}{m} \sum_{i=1}^{m} L_i$ mini-batch stochastic gradient descent

mini-batch size → hyperparamter: tradeoffs between runtime & memory, convergence & generalization

random fluctuations of SGD help to escape saddle points

How to Train Deep Neural Networks Effectively?

typical loss surface:



source

optimization and regularization difficult

- local vs global minima, saddle points, easily overfitting
- many hyperparameters to tune

but many methods to get it working in practice (despite partly patchy theoretical understanding)

- optimization: activation and loss functions, weight initialization, adaptive learning rate (e.g., Adam), learning rate schedulers
- explicit regularization: weight decay, dropout, data augmentation
- implicit regularization: early stopping, batch/layer normalization, SGD

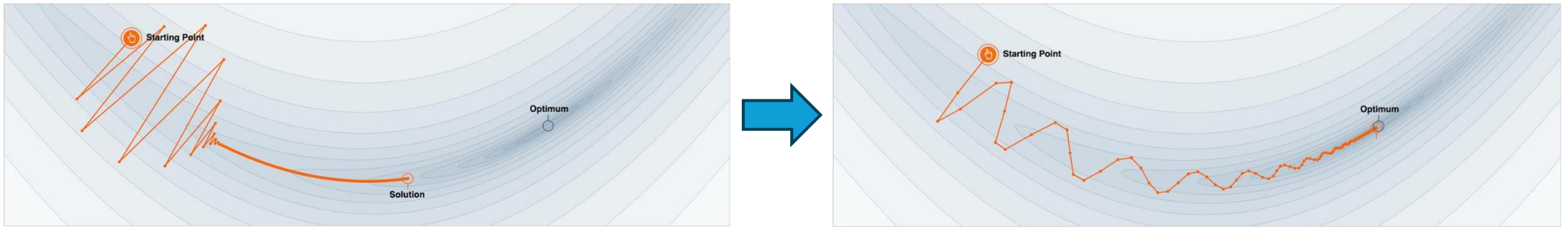
Gradient Descent with Momentum

algorithm:

- estimate gradient: $\boldsymbol{g} = \nabla_{\boldsymbol{w}} \mathcal{L}$
- compute velocity update: $\boldsymbol{v} \leftarrow \alpha\, \boldsymbol{v} - \eta\, \boldsymbol{g}$
- apply weight update: $\boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{v}$

hyperparameter ($0 < \alpha < 1$) specifying exponential decay

→ no bouncing around of weights, escape from local minima and saddle points



Adaptive Learning Rate

better convergence by adapting learning rate for each weight per iteration: lower/higher learning rates for weights with large/small updates

popular methods (g_w denotes component of gradient for **individual weight** w):

- Adagrad: $w \leftarrow w - \frac{\eta}{\sqrt{\sum_{\tau=1}^{t} g_{w,\tau}^2}} g_w$ with t, τ denoting current and past iterations

  (issue: sum in denominator grows with more iterations → can get stuck)
- RMSProp: $w \leftarrow w - \frac{\eta}{\sqrt{v(w)}} g_w$ with $v(w) \leftarrow \gamma\, v(w) + (1-\gamma) g_w^2$
- Adam (Adaptive Moment Optimization): combines RMSProp with momentum

Learning Rate Schedulers

learning rate η in gradient descent typically set to small value (e.g., 0.001)

use momentum and adaptive learning rates to improve optimization

learning rate scheduling as further alternative (can be used on top):

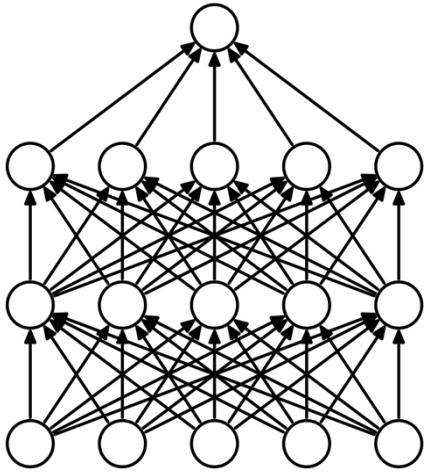
- decay by a certain factor every given number of epochs
- reduce when a metric has stopped improving
- smooth decay by cosine annealing (optionally cyclic with restarts)
- …

Dropout in Neural Networks

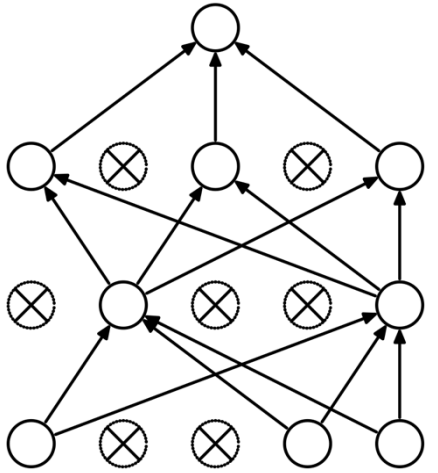
goal: prevent overfitting of large neural networks

randomly drop non-output nodes (along with their connections) during training (not prediction)

→ adaptability: regularizing each hidden node to perform well regardless of which other hidden nodes are in the model



(a) Standard Neural Net



(b) After applying dropout.

source

- for each mini-batch, randomly sample independent binary masks for the nodes
- destroying extracted features rather than input values

Batch Normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

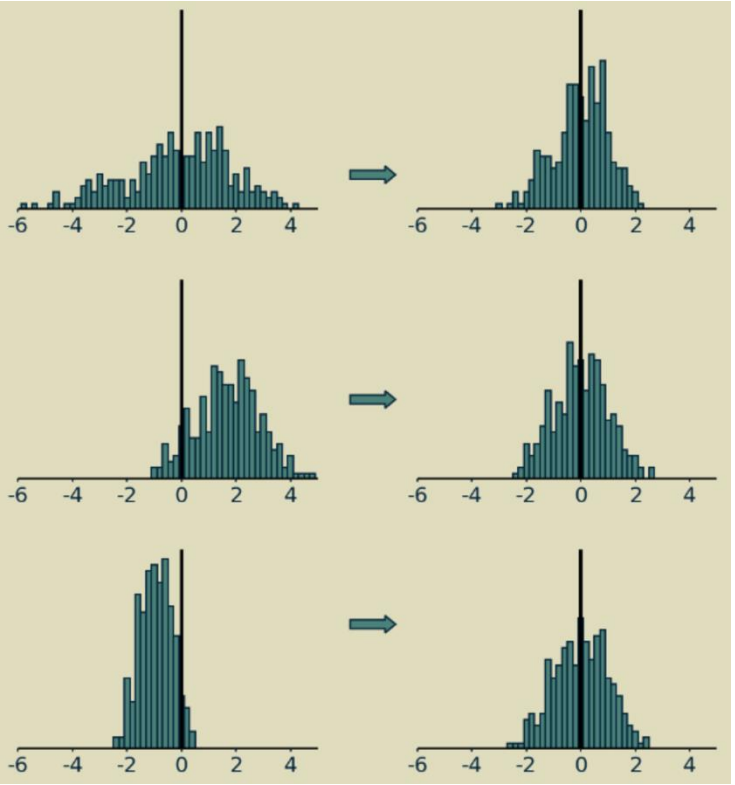
$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

source



source

adaptive reparameterization of inputs to network layer (before or after activation)

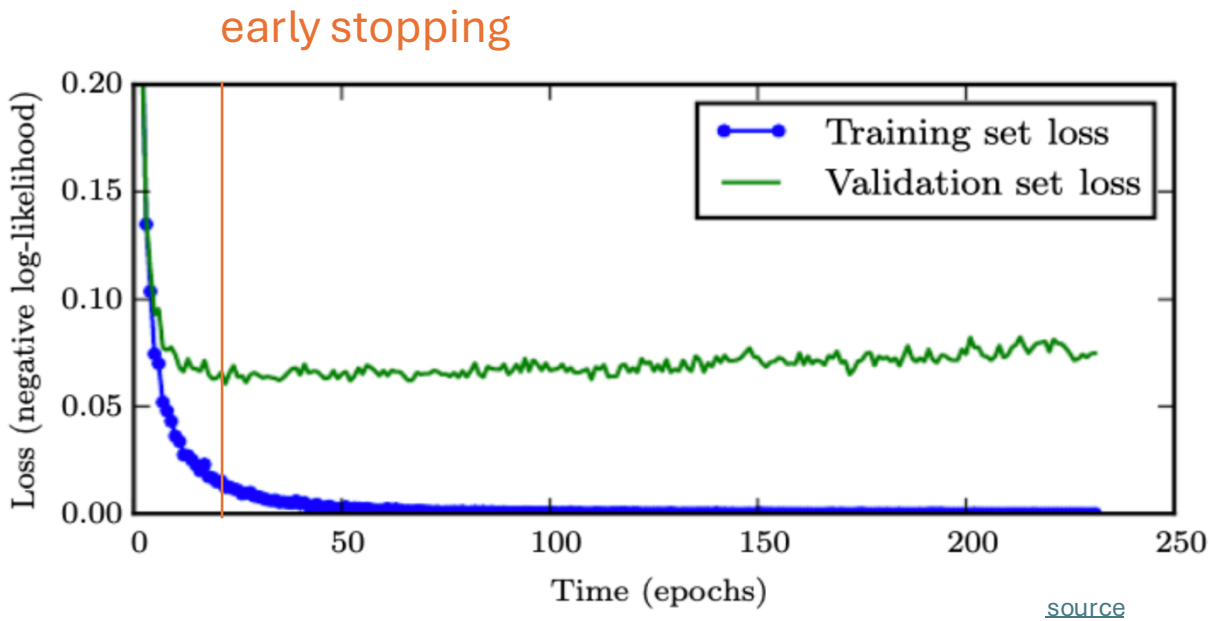
independently for each feature

(implicit) regularization effect

Early Stopping

loss independently measured on validation set

halting training when overfitting begins to occur



source

Tabular vs Unstructured Data

Deep learning excels on unstructured data (images, text, audio), but not necessarily on tabular data.

challenges of tabular data for deep learning:

- heterogeneous features (mixed numerical, categorical, sparse variables)
- limited benefit from feature learning → feature engineering often still required
- weak transfer learning / pretraining opportunities

Tree-based methods (e.g., gradient boosting) naturally handle these properties and often perform better on tabular tasks.

Practice Project

Kaggle competition: Leaf Classification